

课程笔记

CS336 Lecture 16: 对齐与强化学习（一）

Stanford CS336: Language Modeling from Scratch (RLHF、PPO、GRPO 与推理模型)

讲师: Tatsu Hashimoto 笔记: ZCode

2026-07-14



视频作者/频道: Stanford CS336

发布日期: 2025 Spring

视频时长: 80 分钟

视频链接: <https://www.youtube.com/watch?v=46f2QTDB08Q>

Contents

1	导论：从 RLHF 到 RLVR	3
2	RLHF 回顾与收尾	3
2.1	DPO 目标与梯度更新	3
2.2	DPO 的变体：SimPO 与长度归一化	4
2.3	过度优化 (Overoptimization)	4
2.4	RLHF 破坏了模型的校准	5
2.5	本章小结	6
3	RLVR：可验证奖励的强化学习	6
3.1	为什么需要 RLVR	6
3.2	本讲地图	7
4	策略梯度与 PPO	7
4.1	从最简单的策略梯度开始	7
4.2	TRPO：重要性采样修正	8
4.3	PPO：用 clip 代替 KL	9
4.4	PPO 的工程实现：一个“巨兽”	10
4.5	上下文老虎机与奖励整形	11
4.6	广义优势估计 (GAE)	12
4.7	本章小结	12
5	GRPO：去掉价值模型的简化方案	12
5.1	GRPO 的核心思路	12
5.2	组内 z-score 优势	13
5.3	纯在线情形的极简形式	14
5.4	KL 散度的控制变量估计	14
5.5	GRPO 的实现与超参	15
5.6	DeepSeek-Math 的实证结果	16
6	Dr.GRPO：对 GRPO 的数学批评	16
6.1	偏差一：除以标准差	17
6.2	偏差二：长度归一化	17
7	案例研究一：DeepSeek-R1	18
7.1	R1-Zero：纯 RL 的受控实验	19
7.2	“aha moment”与回溯现象	19
7.3	R1 的完整流水线	20
7.4	R1 的性能	22
7.5	R1 的蒸馏	22
7.6	R1 的三大科学贡献	23
8	案例研究二：Kimi K1.5	23
8.1	数据构造	23
8.2	RL 算法：从 DPO 推导出的平方损失	24
8.3	长度奖励：主动压缩 CoT	25
8.4	课程与奖励工程的其他细节	25
8.5	RL 基础设施	26
8.6	Kimi 的训练曲线	26
9	案例研究三：Qwen3	26
9.1	整体流水线	27
9.2	思维模式融合：单模型双模式	27
9.3	Qwen3 的消融：推理 RL vs 通用 RL 的权衡	29

10 总结与展望	30
10.1 核心结论	30
10.2 三个案例的对比	30
10.3 贯穿全讲的几条主线	31

1 导论：从 RLHF 到 RLVR

本讲是后训练（post-training）系列的两讲之二。上一讲围绕 RLHF 与 DPO 展开，本讲首先快速补完 RLHF 的最后几页内容，然后转向近半年最激动人心的方向——可验证奖励的强化学习（RL from Verifiable Rewards, RLVR），它驱动了当前所有主流推理模型（reasoning models）的训练。

本讲的双主线

第一部分（算法）：先把 PPO 讲清楚（上一讲因时长略过的内容），再把 PPO 逐步变形为 GRPO，并深入讨论 GRPO 的实现细节、变体与批评。
第二部分（案例研究）：用三篇代表性的推理模型论文——DeepSeek-R1、Kimi K1.5、Qwen3——展示这些算法在真实工程中如何落地。学完第一部分你就掌握了理解第二部分所需的全部工具。

2 RLHF 回顾与收尾

2.1 DPO 目标与梯度更新

RLHF 的设定是：我们观察到成对偏好数据（pair-wise preferences，即两个回答中哪个更好），希望学到一个语言模型策略 π_θ ，使其最大化某种底层奖励在该偏好数据上的期望。难点在于策略 π_θ 出现在期望的“底部”——你是从语言模型中采样的，而不仅仅是最大化似然。

DPO（Direct Preference Optimization）的核心推导是：先做非参数假设（策略类是所有函数的集合），把奖励重写为策略之比，再代入 Bradley-Terry 目标，于是问题被改写为在一种替代参数化下的监督学习。

DPO – derivation from the RLHF formula

Our goal is to optimize

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [r_\phi(x, y)] - \beta \mathbb{D}_{\text{KL}}[\pi_\theta(y|x) \parallel \pi_{\text{ref}}(y|x)]$$

Assume that the policy π is the set of all policies (nonparametric assumption).
The maximizer is then,

$$\pi_r(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

Solve for the ‘implied reward’

$$r(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x).$$

Stanford
(This is the equivalence also used in the kimi-think paper)

Figure 1: DPO 的目标函数与梯度更新形式¹

DPO 的梯度更新可以写成非常简洁的形式。参数 β 起正则化作用，当奖励估计错误时（隐含奖励不准）更新幅度更大；最终效果就是提高好样本的似然、降低坏样本的似然。

DPO 的核心是非参数假设——把隐含奖励写为策略之比，再代入 Bradley-Terry 目标：

$$r_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

- r_θ ：由策略隐式定义的奖励

¹ 视频画面时间区间：00:01:30 – 00:02:10。

- π_{ref} : 参考策略（通常是 SFT 模型），起锚定作用
- $Z(x)$: 与 y 无关的配分函数（partition function），在 Bradley-Terry 中被消去

代入成对偏好 $y_w \succ y_l$ (w 为偏好项, l 为非偏好项) 后得到 DPO 损失:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right)$$

- σ : sigmoid 函数
- y_w, y_l : 偏好对中的胜者与败者
- β : 控制偏离参考策略的强度 (KL 正则化系数)
- 该损失是纯监督学习——只需成对偏好数据, 无需在线 rollout

一句话总结 RL 算法的本质

几乎所有 RL 算法最终都可以概括为: 上调好东西、下调坏东西 (upweight the good, downweight the bad)。真正的微妙之处只在于两件事——(1) 什么是“好东西”, (2) 该上调多少。DPO 只是众多选择中的一种。

2.2 DPO 的变体: SimPO 与长度归一化

DPO 一度统治了开放模型的后训练流程, 因为它比 PPO 容易调通得多。随后出现了 DPO 变体的“论文洪水”——各种“*PO”。本讲只提两个近期被前沿团队实际采用的:

变体	关键改动
SimPO	(1) 按回答长度对更新幅度归一化; (2) 去掉参考策略 π_{ref}
长度归一化 DPO	保留参考策略, 仅按长度归一化

SimPO 去掉参考策略后, 就失去了 DPO“策略之比”的数学论证, 变成更纯粹的“上调好/下调坏”。其损失按回答长度归一化:

$$\mathcal{L}_{\text{SimPO}}(\theta) = -\log \sigma \left(\frac{\beta}{|y_w|} \log \pi_{\theta}(y_w|x) - \frac{\beta}{|y_l|} \log \pi_{\theta}(y_l|x) - \gamma \right)$$

- 除以 $|y_w|$ 、 $|y_l|$: 按回答长度归一化, 避免长回答被系统性惩罚/奖励
- 无 π_{ref} : 直接用策略自身的对数似然
- γ : 额外的 margin 项 (目标边距)

Tulu 3 (AI2) 在大量实验中尝试了长度归一化 DPO 和 SimPO, 发现它们在某些设置下确实有效。

RL 的结论高度依赖设置

RL 的实证结论非常依赖具体条件: 基座模型、偏好数据、训练环境、SFT 的质量都会显著改变结论。例如 AI2 早期论文发现 PPO 优于 DPO, 但在 Tulu 3 中, 只要 SFT 做得更精细, PPO 和 DPO 的增益都会被吃掉, 只剩长度归一化 DPO 略胜一筹。不要把任何单一实验结果当作普适真理。

2.3 过度优化 (Overoptimization)

过度优化是 RLHF 中最核心、最普遍的问题。本质上它就是“高级版的过拟合”, 但这个名字之所以重要, 是因为它刻画的现象非常危险。

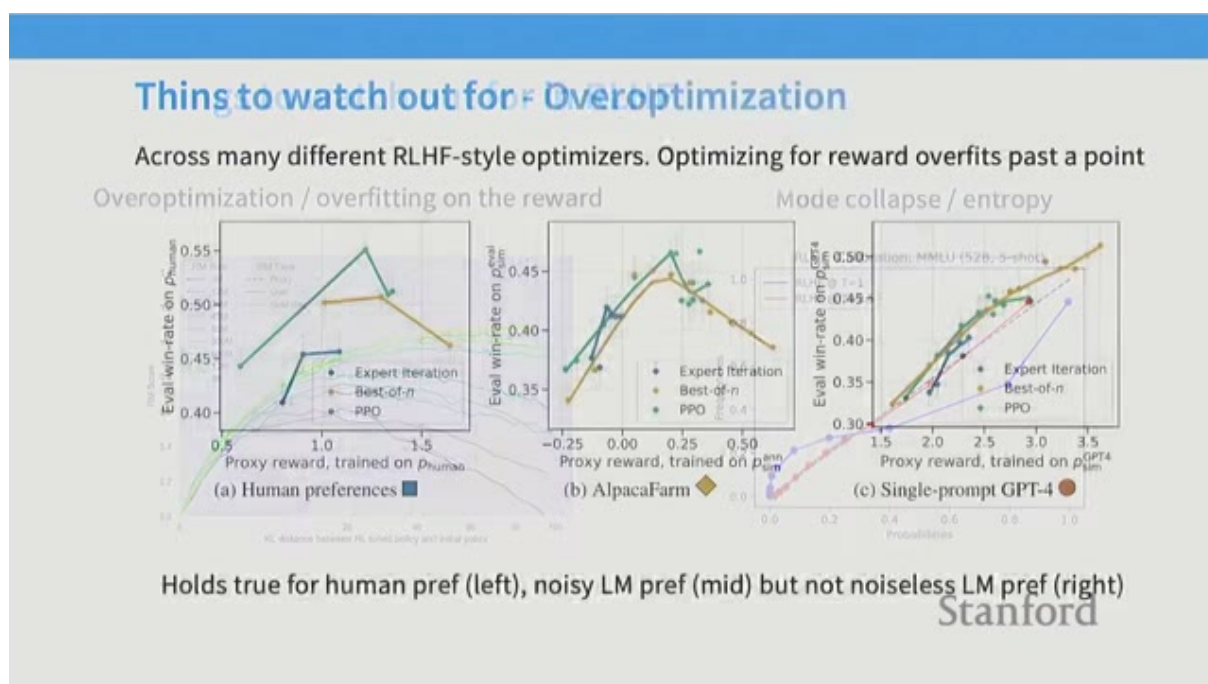


Figure 2: 过度优化：代理奖励持续上升，真实偏好却先升后降²

x 轴是代理奖励（proxy reward，即你拟合的奖励分类器/模型的得分），y 轴是真实偏好胜率（由人类投票或独立的 AI 评测衡量）。一开始两者同向上升，但当代理奖励被优化到一定程度后，真实偏好开始下降——你拟合的奖励模型已经与真实人类偏好发生了分歧。

- r_{proxy} ：在偏好训练集上拟合得到的奖励模型
- r_{true} ：来自真实人类反馈的“上帝视角”奖励
- 两者在期望上度量同一件事，但在有限样本上不一致——这正是机器学习中训练/测试误差鸿沟的翻版

Tatsu 的学生曾做过一项研究：分别用人类偏好、带噪 AI 反馈、无噪 AI 反馈做 RLHF。结果在人类偏好和带噪 AI 反馈上都观察到明显的过度优化曲线，但在无噪 AI 反馈上几乎不出现。这说明过度优化主要源于偏好数据的噪声与复杂性。

过度优化的工程含义

做后训练时必须预期：随着代理奖励上升，真实质量并不单调上升。需要设置独立验证信号（如 held-out 偏好、人工 spot-check）来判断何时停止，否则模型会“优化到一个错误的方向上”。

2.4 RLHF 破坏了模型的校准

另一个重要的副效应是校准（calibration）变差。在预训练和 SFT 阶段，我们做的是概率建模，模型在某种程度上是“校准的”。但 RLHF 后模型只剩“策略”语义，背后不再有清晰的概率分布。

²视频画面时间区间：00:07:00 – 00:07:40。

Things to watch out for - mode collapse

RLHF makes models no longer 'probabilistic models' – no calibration by default

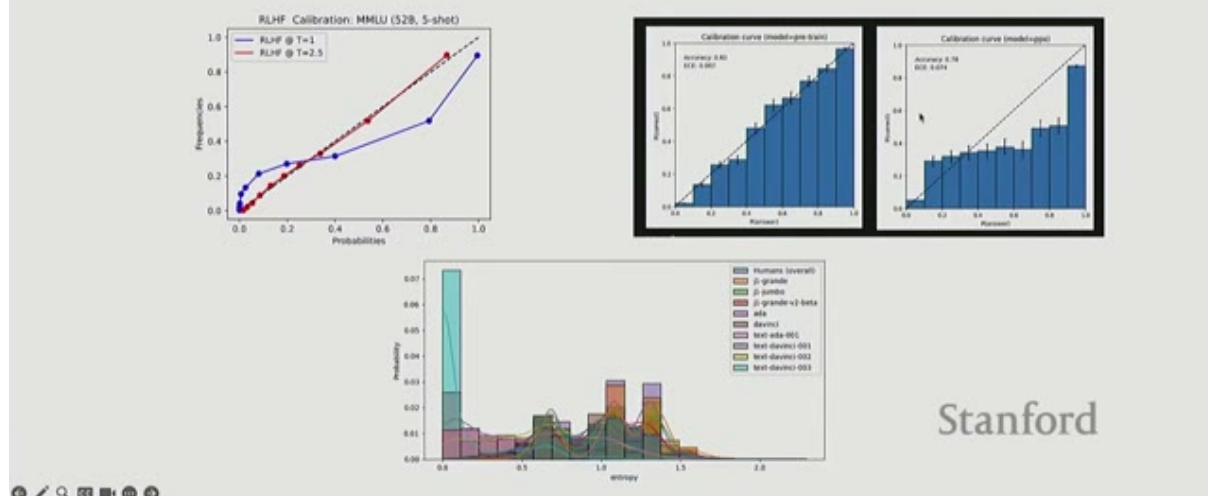


Figure 3: RLHF 后模型在温度 1 下表现出更强的过度自信³

来自 Anthropic、GPT-4 发布以及 Tatsu 组的多篇论文都一致显示：RLHF 模型（尤其在温度 1 下）显著过度自信（overconfident）。如果校准本身不在奖励函数里，模型当然没有理由维持校准——这是“按设计”的结果。

不要把 RLHF 模型当作校准的概率模型

如果你从生成式建模的背景切入，很容易下意识地把 RLHF 模型当作校准的概率预测器。这是危险的。RLHF 改变了输出分布的形状，使得置信度不再可靠地反映真实不确定性。

2.5 本章小结

RLHF 通过 DPO/SimPO 等算法实现了“上调好/下调坏”，但存在两大顽疾：（1）过度优化让代理奖励与真实偏好的关系非单调；（2）破坏概率校准。这些困难正是转向 RLVR 的根本动机。

3 RLVR：可验证奖励的强化学习

3.1 为什么需要 RLVR

人类反馈存在三重困难：容易被 reward hacking、难以大规模采集、难以大规模 RL。过度优化等问题的根源正在于此。

而 RL 真正取得压倒性胜利的领域是 AlphaGo、AlphaFold——这些领域我们知道真实奖励，并能快速、高效、大规模地评估它。如果能找到这样的领域并把 RL 引入语言建模，就能复用 RL 几十年的成功。

³视频画面时间区间：00:08:30 – 00:09:00。

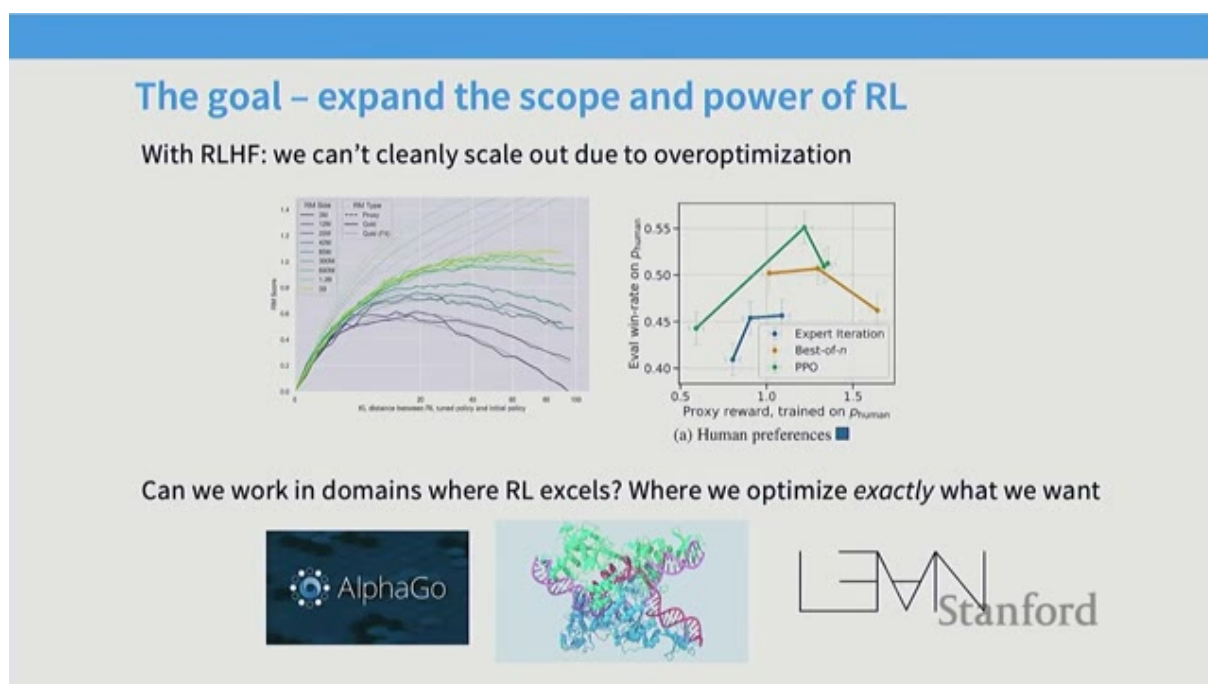


Figure 4: 从 RLHF（人类偏好）到 RLVR（可验证奖励）的范式转移⁴

RLHF 与 RLVR 的对照

RLHF：奖励来自人类偏好（噪声大、难采集、易被 hack、导致过度优化）。RLVR：奖励来自可程序化验证的真值（数学答案对错、代码测试通过），干净、可大规模评估、几乎不可能被 reward hacking。范式从“讨好人类”转向“解对题目”。

RLVR 的核心心法

与其用“用户是否喜欢这个机器人”这一难优化、易被 hack 的目标，不如选择一个我们知道真实奖励、且能高效评估的领域（如数学：答案对就是对、错就是错），然后把 RL 的全部武器搬过来。这正是从 ChatGPT-3.5 时代迈向 o1 时代的关键思路转变。

3.2 本讲地图

部分	内容
第一部分	PPO（详细）→ GRPO（简化）→ GRPO 变体与 Dr.GRPO 批评
第二部分	DeepSeek-R1（R1-Zero、R1）、Kimi K1.5、Qwen3 三个案例研究

4 策略梯度与 PPO

4.1 从最简单的策略梯度开始

理解 PPO 的最佳路径是从最简单的东西出发，再一步步往下走。最简单的就是策略梯度（policy gradient）：在当前策略 π_θ 下最大化期望奖励，对其求梯度可得：

⁴视频画面时间区间：00:12:55 – 00:13:25。

Recap: PPO in theory

A brief (and high level) intro to the various ideas in PPO..

Attempt 1: Policy gradients (variances are too high)

$$\nabla_{\theta} E_{p_{\theta}}[R(z)] = E_{p_{\theta}}[R(z) \nabla_{\theta} \log p_{\theta}(z)]$$

Attempt 2: TRPO (Linearize the problem around the current policy)

$$\begin{aligned} \underset{\theta}{\text{maximize}} \quad & \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] \\ \text{subject to} \quad & \hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]] \leq \delta. \end{aligned}$$

Attempt 3: PPO (Clip the ratios at some eps)

$$L(s, a, \theta_k, \theta) = \min \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s, a), \text{clip} \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta_k}}(s, a) \right)$$

Figure 5: 策略梯度：期望奖励对参数求梯度⁵

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{z \sim \pi_{\theta}} [r(z) \nabla_{\theta} \log \pi_{\theta}(z)]$$

- z : 从策略 π_{θ} 采样的一条轨迹 (rollout)
- $r(z)$: 该轨迹的奖励
- $\nabla_{\theta} \log \pi_{\theta}(z)$: 对数似然的得分函数 (score function)
- 根据奖励的符号，梯度会相应地提高或降低该样本的概率

策略梯度的两个低效之处：第一，它是纯 on-policy 的——每次梯度步都要从 π_{θ} 重新采样并立即更新；第二，RL 最贵的部分是 rollout（运行语言模型生成样本），而 on-policy 不允许我们对同一批样本多次更新。

更根本地，我们想要最大化的原始目标是策略下的期望奖励：

$$J(\theta) = \mathbb{E}_{z \sim \pi_{\theta}} [r(z)]$$

- π_{θ} : 参数为 θ 的策略（语言模型）
- $r(z)$: 轨迹 z 的奖励
- 这是一个策略上的期望——策略本身既是被优化的对象，又出现在期望的分布中，这正是它难于优化的原因

4.2 TRPO: 重要性采样修正

为了减少采样次数，希望允许“更新变旧”——从旧的 $\pi_{\theta_{\text{old}}}$ 采样、做多次更新，但仍保持策略梯度的有效性。做法是重要性采样 (importance sampling) 修正，并用 KL 约束把新策略限制在旧策略附近，避免奖励估计失真。

⁵视频画面时间区间：00:15:00 – 00:15:30。

Recap: PPO in theory

A brief (and high level) intro to the various ideas in PPO..

Attempt 1: Policy gradients (variances are too high)

$$\nabla_{\theta} E_{p_{\theta}}[R(z)] = E_{p_{\theta}}[R(z) \nabla_{\theta} \log p_{\theta}(z)]$$

Attempt 2: TRPO (Linearize the problem around the current policy)

$$\begin{aligned} \underset{\theta}{\text{maximize}} \quad & \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] \\ \text{subject to} \quad & \mathbb{E}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]] \leq \delta. \end{aligned}$$

Attempt 3: PPO (Clip the ratios at some eps)

$$L(s, a, \theta_k, \theta) = \min \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s, a), \text{clip} \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta_k}}(s, a) \right)$$

Figure 6: TRPO: 重要性采样 + KL 约束, 允许 off-policy 更新⁶

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{z \sim \pi_{\theta_{\text{old}}}} \left[\frac{\pi_{\theta}(z)}{\pi_{\theta_{\text{old}}}(z)} A(z) \nabla_{\theta} \log \pi_{\theta}(z) \right]$$

- $\frac{\pi_{\theta}(z)}{\pi_{\theta_{\text{old}}}(z)}$: 重要性采样比, 修正分布偏移
- $A(z)$: 优势函数 (advantage), 是 $r(z)$ 的低方差版本
- TRPO 用 KL 散度硬约束保证 $\pi_{\theta} \approx \pi_{\theta_{\text{old}}}$

4.3 PPO: 用 clip 代替 KL

PPO 是在 TRPO 之上再走一步: 不做 KL 散度, 而是直接截断 (clip) 优势函数。当策略偏离过远时, 奖励会被截断在 $1 - \epsilon$ 或 $1 + \epsilon$ 处, RL 算法再没有动机让策略跑得更远——这是一种软约束。

⁶视频画面时间区间: 00:16:35 - 00:17:00。

Recap: PPO in theory

A brief (and high level) intro to the various ideas in PPO..

Attempt 1: Policy gradients (variances are too high)

$$\nabla_{\theta} E_{p_{\theta}}[R(z)] = E_{p_{\theta}}[R(z) \nabla_{\theta} \log p_{\theta}(z)]$$

Attempt 2: TRPO (Linearize the problem around the current policy)

$$\begin{aligned} & \underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] \\ & \text{subject to} \quad \hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]] \leq \delta. \end{aligned}$$

Attempt 3: PPO (Clip the ratios at some eps)

$$L(s, a, \theta_k, \theta) = \min \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s, a), \text{clip} \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta_k}}(s, a) \right)$$

Figure 7: PPO clip 目标：截断重要性比，自然约束策略漂移⁷

$$L^{\text{CLIP}}(\theta) = \mathbb{E} [\min (\rho_t(\theta) A_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)]$$

- $\rho_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$: 重要性比
- ϵ : 截断范围，典型值 0.2，即允许的似然比区间为 $[0.8, 1.2]$
- A_t : 优势函数，由价值函数 (value function) 计算，用于方差缩减

PPO 需要第二个神经网络

PPO 的优势 A_t 需要一个价值网络来估计期望回报以降低梯度方差。这个价值网络通常和策略网络一样大——这意味着 GPU 显存翻倍。这是后来 GRPO 试图摆脱 PPO 的核心动机之一。

4.4 PPO 的工程实现：一个“巨兽”

PPO 在概念上不复杂，但工程上极其繁琐。有一篇著名的博客《37 implementation details of PPO》⁷以及专门论文论证“实现细节比算法本身更重要”——甚至有些细节严格意义上违反了策略梯度推导，却反而效果更好。

⁷ 视频画面时间区间：00:17:15 – 00:17:35。

PPO in practice

PPO outer loop. Invoke an inner loop to optimize the loss over some rollouts.

```
def step_with_rollouts(self, rollouts):
    """Based on fixed rollouts, run PPO for multiple epochs."""
    assert isinstance(self.optimizer, AcceleratedOptimizer), (
        "optimizer must be pushed through 'accelerator.prepare'. "
        "Otherwise the 'accelerator.accumulate' context manager won't correctly disable 'zero_grad' or 'step'."
    )
    rollouts_dataloader = self.get_rollouts_dataloader(rollouts=rollouts)
    stats_list = []
    for epoch_idx in range(self.args.noptepochs):
        for batch_idx, rollouts_batch in tqdm.tqdm(
            enumerate(rollouts_dataloader, 1), disable=not self.accelerator.is_main_process, desc="gradstep"
        ):
            with self.accelerator.accumulate(self.policy):
                ppo_loss, stats_for_this_step = self.compute_loss(rollouts_batch)
                self.accelerator.backward(ppo_loss)
                if self.accelerator.sync_gradients:
                    # Gradient norm almost blows up at some point, but stabilizes eventually, even w/o clipping.
                    if self.args.max_grad_norm is not None:
                        self.accelerator.clip_grad_norm(self.policy.parameters(), self.args.max_grad_norm)
                stats_for_this_step["loss/grad_norm"] = self._compute_grad_norm()
                stats_list.append(stats_for_this_step)
            self.optimizer.step()
            self.optimizer.zero_grad(set_to_none=True)
    return common.merge_dicts(stats_list, torch.stack) # list of dict => dict: str => 1-D tensor
```

Stanford

https://github.com/tatsu-lab/alpaca_farm/blob/30717ddae735365de756ee2085191b491a71788d/src/alpaca_farm/rl/ppo_trainer.py

Figure 8: 早期 RLHF 的 PPO 实现示意图：奖励模型、价值模型、GAE 一应俱全⁸

标准 PPO 实现包含以下组件：

组件	作用	实现要点
外层训练循环	标准 LM 梯度更新	rollout → 算 loss → backward + grad clip
价值函数 loss	拟合期望回报	与策略同步更新
PPO clip loss	截断重要性比	$\epsilon = 0.2$
rollout 模块	采样 + 算 logprob	价值/奖励/策略分词器不同需重分词
reward shaping	构造 per-token 奖励	KL 按 token、真奖励只在终止 token
GAE	方差缩减	可设 $\gamma = \lambda = 1$ 退化为 baselined PG

4.5 上下文老虎机与奖励整形

一个关键观察：语言模型的 RL 在某种意义上是上下文老虎机（contextual bandit）——给定 prompt 生成一个回答，立即获得奖励，没有状态转移、没有环境探索。这种结构使得奖励可以这样塑造：

- 终止奖励（terminal reward）：任务是否成功（如数学答案对错），只在最后一个 token 上给出
- per-token 奖励：KL 正则项按 token 计算，作为“持续的小惩罚”

KL 的实现细节陷阱

PPO 实现中常按 token 加 KL 惩罚，但代码中往往把 log-似然比在为负时截断到 0 以避免数值不稳定——这其实已经不再是严格意义的 KL 了，而是一种近似。这种“工程近似”在 RLHF 代码中非常普遍。

更精确地说，per-token 的 KL 惩罚项形如：

$$\hat{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(o_t | s_t)}{\pi_{\theta}(o_t | s_t)} - \log \frac{\pi_{\text{ref}}(o_t | s_t)}{\pi_{\theta}(o_t | s_t)} - 1$$

该 per-token 惩罚在所有非终止 token 上累加，而任务奖励（如答案对错）只在最后一个 token 上给出：

$$R_{\text{total}}(z) = \mathbb{I}[\text{correct}] + \sum_{t=1}^{|z|} \beta \cdot \hat{D}_{\text{KL}}^{(t)}$$

⁸视频画面时间区间：00:20:30 – 00:21:30。

4.6 广义优势估计 (GAE)

策略梯度的方差通常很高，需要尽可能的方差缩减。与其直接用 $r(z)$ 乘梯度，可以用一个折扣优势估计 A_t 替代，其形式由 GAE (Generalized Advantage Estimation) 给出：

PPO in practice – generalized advantage estimate

Instead of reward, we use advantages

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} := \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V \quad \text{where} \quad \delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t).$$

```
00 = def _estimate_advantage(self, rewards: Tensor, values: Tensor) -> Dict[str, Tensor]:
01     """Generalized advantage estimation.
02
03     Reference:
04     https://arxiv.org/abs/1506.02438
05     """
06     if self.args.whiten_rewards:
07         rewards = torch_ops.whiten(rewards, shift_mean=False)
08     lastgaelam = 0
09     advantages_reversed = []
10     gen_length = self.args.response_len
11     for t in reversed(range(gen_length)):
12         nextvalues = values[t, t+1] if t < gen_length - 1 else 0.0
13         delta = rewards[t, t] + self.args.gamma * nextvalues - values[t, t]
14         lastgaelam = delta + self.args.gamma * self.args.lam * lastgaelam
15         advantages_reversed.append(lastgaelam)
16     advantages = torch.stack(advantages_reversed[::-1], dim=1)
17     returns = advantages + values
18     advantages = torch_ops.whiten(advantages, shift_mean=True)
19     return dict(returns=returns, advantages=advantages)
```

Funny detail – this is a bandit problem and gamma=lambda=1 works – this is the reward-to-go vs the value

Figure 9: GAE: 通过 γ 、 λ 在偏差与方差间权衡⁹

$$A_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

- δ_t : 时序差分 (TD) 误差
- γ : 折扣因子，控制对未来回报的重视
- λ : GAE 参数，在偏差 ($\lambda \rightarrow 0$) 与方差 ($\lambda \rightarrow 1$) 间权衡
- $V(s)$: 价值函数估计
- 一个有趣的事实：取 $\gamma = \lambda = 1$ ，GAE 退化为 baselined 策略梯度 ($R - V$)，往往也能很好地工作

4.7 本章小结

PPO = 策略梯度 + 重要性采样修正 + clip 软约束 + 价值函数 + GAE。它在通用 RL 上极其成功 (OpenAI 的 Dota 机器人、各种 toy 环境)，但在语言模型上是一个“巨兽”——37 个实现细节、需要与策略等大的价值网络、容易翻车的 KL 近似。这正是后续寻找替代方案的动机。

5 GRPO：去掉价值模型的简化方案

5.1 GRPO 的核心思路

人们想要：能像 PPO 那样适用通用 RL 设置、但不要复杂的实现、更重要的不要价值模型的算法。DPO 不合适——它针对成对 Bradley-Terry 偏好，不适合“数学答案对错”这种无成对结构的场景；它本质上是离线算法。

于是就有了 GRPO (Group Relative Policy Optimization)。GRPO 极其简单：从 PPO 出发，保留 clip 与策略更新的思想，但彻底删掉 GAE，换成一个简单得多的优势估计。

⁹视频画面时间区间：00:26:15 – 00:26:45。

New kid on the block: GRPO

What's GRPO?

- Start with PPO (many parts are similar)
- Remove the value function / advantage computation
- Calculate the advantage as “z-score within group”

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)] \frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_\theta \parallel \pi_{ref}) \right), \quad (1)$$

$$\mathbb{D}_{KL}(\pi_\theta \parallel \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \quad (2)$$

where ϵ and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \dots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}. \quad (3)$$

(PPO for reference)

$$\min \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)} A^{v_{\pi_{\theta_{old}}}(s, a)}, \text{clip} \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{v_{\pi_{\theta_{old}}}(s, a)} \right)$$

Stanford

In the online case (rollout+immediate update), this is just policy gradient with group normalized rewards

Figure 10: GRPO: 用组内 z-score 替代 GAE, 去掉价值模型¹⁰

5.2 组内 z-score 优势

New kid on the block: GRPO

What's GRPO?

- Start with PPO (many parts are similar)
- Remove the value function / advantage computation
- Calculate the advantage as “z-score within group”

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)] \frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_\theta \parallel \pi_{ref}) \right), \quad (1)$$

$$\mathbb{D}_{KL}(\pi_\theta \parallel \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \quad (2)$$

where ϵ and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \dots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}. \quad (3)$$

(PPO for reference)

$$\min \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)} A^{v_{\pi_{\theta_{old}}}(s, a)}, \text{clip} \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{v_{\pi_{\theta_{old}}}(s, a)} \right)$$

Stanford

In the online case (rollout+immediate update), this is just policy gradient with group normalized rewards

Figure 11: GRPO 的组 (group) : 同一问题的 G 个候选回答¹¹

GRPO 的优势定义为:

$$A_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}$$

¹⁰视频画面时间区间: 00:29:25 – 00:29:55。

¹¹视频画面时间区间: 00:30:00 – 00:30:45。

- r_i : 第 i 个回答的奖励
- mean、std: 同一问题的 G 个回答的组内均值与标准差
- 这是对组内奖励的 **z-score** (标准化得分)
- ”组”对语言模型 RL 是天然对象: 一个输入问题 (如一道数学题) + G 个候选回答构成一组

带 clip 的 GRPO 完整目标函数为:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \min(\rho_i A_i, \text{clip}(\rho_i, 1 - \varepsilon, 1 + \varepsilon) A_i) - \beta \hat{D}_{\text{KL}} \right]$$

- $\rho_i = \pi_{\theta}(o_i|q) / \pi_{\theta_{\text{old}}}(o_i|q)$: 重要性采样比
- A_i : 组内 z-score 优势
- $\hat{D}_{\text{KL}}$: 策略与参考策略的 KL 估计
- 与 PPO 相比, 只是把价值函数算出的 GAE 换成了组内 z-score

baselining 是策略梯度降低方差的核心技巧。经典结果表明, 可以减去任意不依赖当前样本的基线 B 而不改变梯度期望:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{z \sim \pi_{\theta}} [(r(z) - B) \nabla_{\theta} \log \pi_{\theta}(z)]$$

- B : 基线, 只要与 z 独立 (或为常数), 在策略上求和为 0
- 减去合适的 B 可大幅降低梯度方差而不引入偏差
- GRPO 的组内均值正是这样一种自适应基线

”组”的工程含义

在工程实现中, 你会同时处理多个问题, 每个问题采样 G 个回答构成一组; baseline 只在组内做 (同一问题的 G 个回答之间归一化), 跨问题之间不做 baselining, 只共享策略更新。这种结构天然适配 batch 训练——一个 batch 由若干个”问题 $\times G$ 回答”的组拼成。

为什么组内 baseline 是合理的

不同问题的难度不同——简单的题平均奖励高、难题平均奖励低。组内均值天然捕捉了”问题难度”, 作为每个样本自身的基线 (baseline) 被减掉。这正是”policy gradient with leave-one-out baseline”思想的简化 (去掉标准差项即为 leave-one-out)。baseline 只在同一个问题内部做, 跨问题不做 baselining, 只共享策略更新。

5.3 纯在线情形的极简形式

如果只做一步纯在线更新 (即不重复使用 rollout), 所有的 clip 机制都可以丢掉, 剩下的就是纯粹的策略梯度——上调好东西、下调坏东西, 只不过乘上去的权重变成了 A_i 。这是一个极其简单的算法。

5.4 KL 散度的控制变量估计

GRPO 中的 KL 散度 D_{KL} 估计有点非标准。自然的 KL 估计是对 log 比求平均, 但 GRPO 用了带两个额外项的形式:

New kid on the block: GRPO

What's GRPO?

- Start with PPO (many parts are similar)
- Remove the value function / advantage computation
- Calculate the advantage as “z-score within group”

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \mathcal{D}_{KL}(\pi_\theta \parallel \pi_{ref}) \right), \quad (1)$$

$$\mathcal{D}_{KL}(\pi_\theta \parallel \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \quad (2)$$

where ϵ and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \dots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}. \quad (3)$$

(PPO for reference)

$$\min \left(\frac{\pi_\theta(o|s)}{\pi_{\theta_{old}}(o|s)} A^{v_{\pi_{\theta_{old}}}}(s, a), \text{clip} \left(\frac{\pi_\theta(o|s)}{\pi_{\theta_{old}}(o|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{v_{\pi_{\theta_{old}}}}(s, a) \right)$$

Stanford

In the online case (rollout+immediate update), this is just policy gradient with group normalized rewards

Figure 12: GRPO 的 KL 估计：控制变量（control variate）方案以降低方差¹²

$$D_{KL} \approx \frac{\pi_{ref}(o|x)}{\pi_\theta(o|x)} - \log \frac{\pi_{ref}(o|x)}{\pi_\theta(o|x)} - 1$$

- 第一项 $\frac{\pi_{ref}}{\pi_\theta}$ 在对 π_θ 求期望时与第三项 -1 抵消
- 这构成一个控制变量（control variate），在不改变期望的同时降低方差
- 这是估计 KL 的一个更“漂亮”的方式，值得在需要从样本估计 KL 时借鉴

5.5 GRPO 的实现与超参

GRPO 的实现几乎逐行对应其公式。外层循环：对每个 rollout 算奖励、组内归一化（均值和方差）、算 KL（实现中按 sequence 算，重型实现会更细）、对 loss 做梯度更新。优势计算几乎逐行照抄公式，唯一的小差异是除以标准差时加一个小的 10^{-4} 防 0：

$$A_i = \frac{r_i - \bar{r}}{\sqrt{\frac{1}{G} \sum_{j=1}^G (r_j - \bar{r})^2 + \epsilon}}, \quad \epsilon = 10^{-4}$$

- $\bar{r} = \frac{1}{G} \sum_j r_j$ ：组内均值
- 分母加 ϵ ：防止当组内奖励完全相同（标准差为 0）时除零崩溃
- 这是几乎所有 RL 归一化代码都会做的标准技巧

作业提示

在 A5 中你将实现 GRPO。优势计算需要加一个小的 epsilon（如 10^{-4} ）防止除以 0；KL 在不同实现中可以是 per-token 或 per-sequence；价值/奖励/策略模型若分词器不同需要重分词。

¹²视频画面时间区间：00:32:45 – 00:33:15。

5.6 DeepSeek-Math 的实证结果

GRPO 来自 DeepSeek-Math 论文。该论文比较了多种方法：

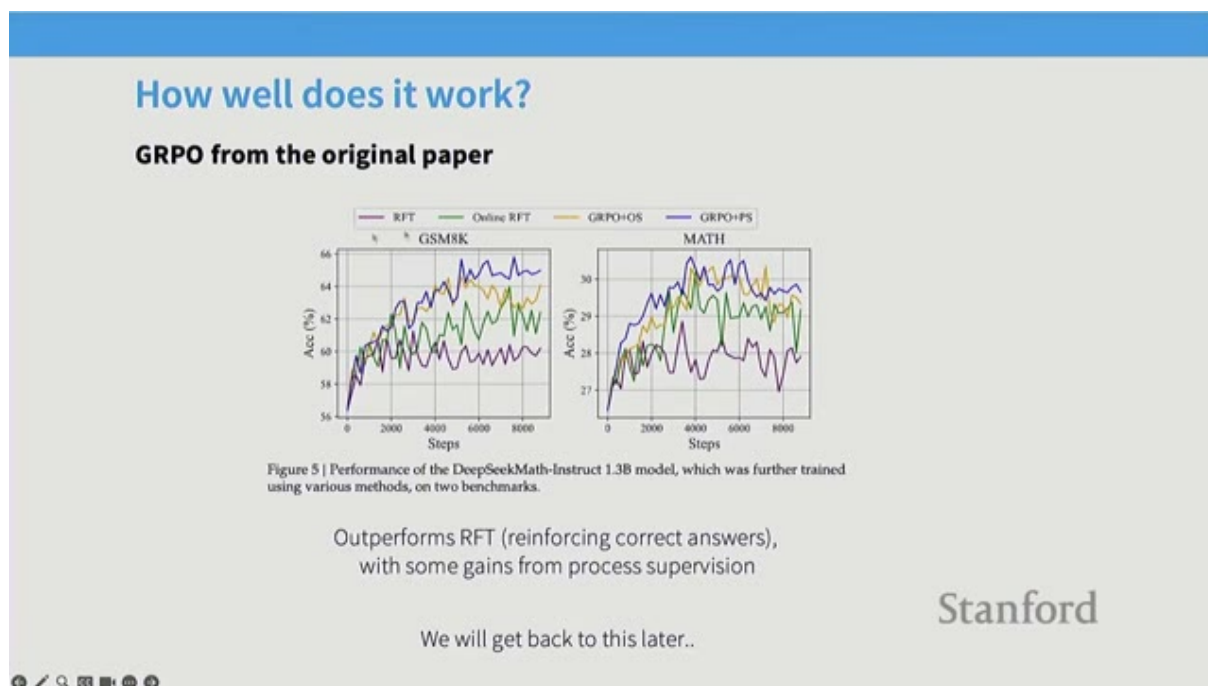


Figure 13: DeepSeek-Math: RFT、GRPO（结果奖励）与 GRPO（过程奖励）对比¹³

- RFT/Online RFT：只对答对的样本做 SFT，是较弱的基线
- GRPO + outcome reward（黄色）：只用答案对/错的二元奖励
- GRPO + process reward（蓝色）：对每步推理打分（PRM），论文论证 PRM 更好
- 结论：GRPO 有效，效果显著

PRM 的“翻转”

有意思的是：DeepSeek-Math 阶段他们对 PRM 信心满满（蓝线最优），但到了 R1 论文，他们反而放弃了 PRM，回到了纯结果奖励。这一翻转贯穿整个推理模型的发展史，值得反复体会。

6 Dr.GRPO：对 GRPO 的数学批评

回到策略梯度定理。一条经典结果是：可以减去任意不依赖当前样本的基线 $B(s)$ 而保持梯度期望不变——因为 $B(s)$ 在策略上求和为 0。这叫做 **baselining**，是降低方差的标准工具。

GRPO 的 A_i 是基线吗？减去组内均值勉强符合（只要不含 r_i 自身），但除以标准差这一步在数学上是有问题的——它不在策略梯度定理所允许的操作内。

¹³视频画面时间区间：00:35:00 – 00:35:40。

Length biases of GRPO

What do these terms do? **Stdev** – upweights too easy or hard questions.

Length normalization:

Response-level length bias: This arises from dividing by $|\mathbf{o}_i|$. For positive advantages ($\hat{A}_{i,j} > 0$, indicating a correct response), this bias results in greater gradient updates for shorter responses, leading the policy to favor brevity in correct answers. Conversely, for negative advantages ($\hat{A}_{i,j} < 0$, indicating an incorrect response), longer responses are penalized less due to their larger $|\mathbf{o}_i|$, causing the policy to prefer lengthier responses among incorrect ones.

What does the fix do?

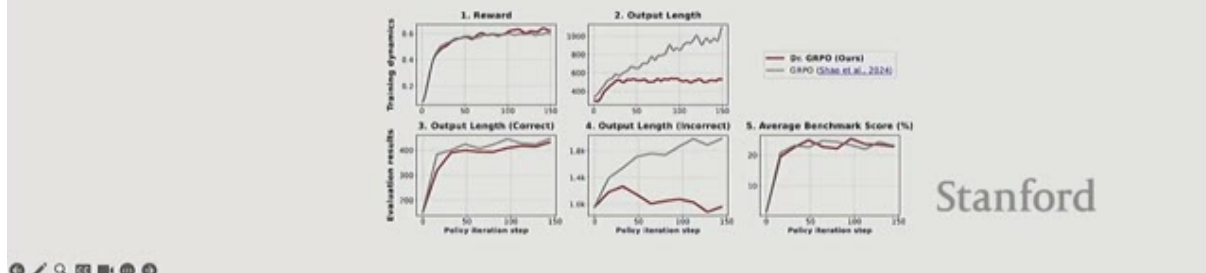


Figure 14: Dr.GRPO: GRPO 的两个数学“不合规”之处¹⁴

6.1 偏差一：除以标准差

当标准差小时（问题太容易，奖励全为 1；或太难，全为 0），优势会被放大——算法会过度优化那些“太容易或太难”的组。这与理想的“课程效应”背道而驰：我们真正想喂给模型的是难度适中、能学到东西的样本。

folk theorem 的直觉

RL 算法最想拿到的是“模型有时做对、有时做错”的样本——这样才有学习信号。除以标准差反而把优化压力推向两个极端（要么全对、要么全错），这些样本几乎没有可学的梯度方向，可能拖慢收敛，至少在数学上破坏了策略梯度的有效性。

6.2 偏差二：长度归一化

GRPO 还把奖励按输出长度归一化。这看起来无害，却引入了糟糕的激励：

¹⁴视频画面时间区间：00:39:25 – 00:40:00。

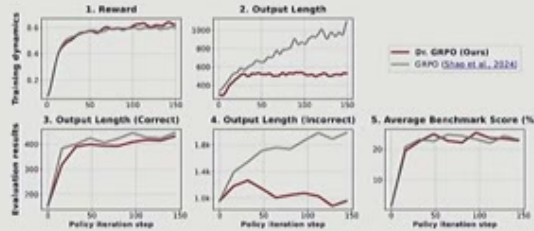
Length biases of GRPO

What do these terms do? **Stdev** – upweights too easy or hard questions.

Length normalization:

Response-level length bias: This arises from dividing by $|\mathbf{o}_i|$. For positive advantages ($\hat{A}_{i,j} > 0$, indicating a correct response), this bias results in greater gradient updates for shorter responses, leading the policy to favor brevity in correct answers. Conversely, for negative advantages ($\hat{A}_{i,j} < 0$, indicating an incorrect response), longer responses are penalized less due to their larger $|\mathbf{o}_i|$, causing the policy to prefer lengthier responses among incorrect ones.

What does the fix do?



Stanford

Figure 15: Dr.GRPO 修正后：输出长度不再无限增长¹⁵

- 答错时：得到负奖励，最优策略是把回答拉到无限长以稀释负奖励
- 答对时：得到正奖励，最优策略是把回答压到最短以最大化单位长度奖励
- 结果：模型产生“答错时疯狂胡说、答对时惜字如金”的扭曲激励

Dr.GRPO 的作者主张同时去掉这两个偏差。实验显示：在 GSM8K 等 toy 任务上，修正后能达到相同奖励，但输出长度不再无限增长，而是稳定在一个水平。

修正后的 Dr.GRPO 优势定义为：

$$A_i^{\text{Dr.GRPO}} = r_i - \text{mean}(r_1, \dots, r_G)$$

- 去掉了除以标准差，只保留减去均值
- 这回到了合法的 leave-one-out baseline，符合策略梯度定理的要求
- 不再过度放大“太易/太难”的样本

对“长 CoT 是必然”的质疑

一个引人深思的假说：我们看到的许多超长思维链（CoT），可能并非推理能力的内在需求，而是 GRPO 实现细节的副产物。这一假说虽未完全证实，但 Dr.GRPO 的实验提供了相当有力的旁证。

7 案例研究一：DeepSeek-R1

R1 是一篇引发社会现象的 arXiv 论文——一度让 Nvidia 蒸发近五亿美元市值。它的贡献在于：用极其简单的配方复现了 OpenAI o1 的全部定性能力。

- 性能：全面逼近 o1
- 配方：没有搜索、没有 PRM、没有花哨的机制——纯 RL
- 科学洞见：SFT 与 RL 之间交互的许多细节

¹⁵ 视频画面时间区间：00:40:25 – 00:41:00。

7.1 R1-Zero: 纯 RL 的受控实验

R1-Zero 是 R1 的”研究设置”。它直接拿预训练+mid-trained 的 DeepSeek-V3 基座（不做任何 RLHF 或指令微调），扔进数学 RL 循环。

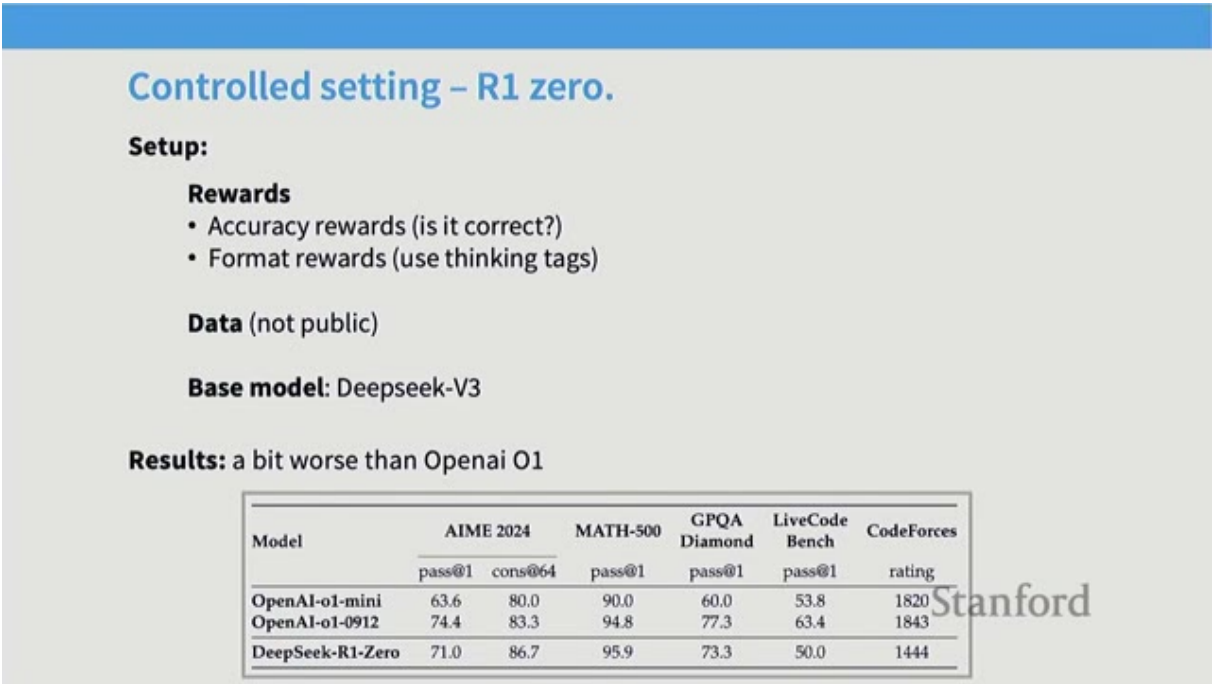


Figure 16: R1-Zero: 基座模型直接做 RL，无 SFT¹⁶

要素	设置
基座	DeepSeek-V3（pre-trained + mid-trained）
数据	类数学任务（数据未公开）
奖励 1	accuracy reward：答案对/错，二元
奖励 2	format reward：强制把思维链放进 <think></think> 标签

R1-Zero 的总奖励是 accuracy 与 format 两项之和：

$$R(o, q) = R_{\text{accuracy}}(o, q) + R_{\text{format}}(o)$$

- $R_{\text{accuracy}} \in \{0, 1\}$: 答案是否正确（由规则判题器或模型判等给出）
- $R_{\text{format}} \in \{0, 1\}$: 回答是否包含合法的 <think></think> 标签
- 这是一种纯可验证奖励——无任何人类偏好参与

format reward 看似无关，实则关键

format reward 表面是个无关紧要的格式约束，但据许多论文和工程实践反馈：它其实是让整套推理 RL 跑通的关键部件之一。没有它，模型很难稳定地产出可被后续流程利用的长 CoT。

7.2 ”aha moment” 与回溯现象

R1-Zero 训练中出现两个被大肆宣传的现象：

¹⁶视频画面时间区间：00:46:15 – 00:46:45。

But maybe a bit overstated

From some follow-up analyses in Dr. GRPO..

Length due to
biased objective?

$$\begin{aligned} \text{GRPO} \\ \frac{1}{C} \sum_{c=1}^C \frac{1}{M} \sum_{m=1}^M \left\{ \min \left[\frac{\pi_{\theta}(\mathbf{a}_{c,m} | \mathbf{q}_{c,m})}{\pi_{\theta_{\text{old}}}(\mathbf{a}_{c,m} | \mathbf{q}_{c,m})} A_{c,m}, \exp \left(\frac{\pi_{\theta}(\mathbf{a}_{c,m} | \mathbf{q}_{c,m})}{\pi_{\theta_{\text{old}}}(\mathbf{a}_{c,m} | \mathbf{q}_{c,m})} (1 - \epsilon, 1 + \epsilon) A_{c,m} \right) \right] \right\}, \\ \text{where } A_{c,m} = \frac{R(\mathbf{q}, \mathbf{a}_c) - \text{mean}(\{R(\mathbf{q}, \mathbf{a}_1), \dots, R(\mathbf{q}, \mathbf{a}_C)\})}{\max(\{R(\mathbf{q}, \mathbf{a}_1), \dots, R(\mathbf{q}, \mathbf{a}_C)\})}. \end{aligned}$$
$$\begin{aligned} \text{Dr. GRPO} \\ \text{GRPO Done Right (without bias)} \\ \frac{1}{C} \sum_{c=1}^C \frac{1}{M} \sum_{m=1}^M \left\{ \min \left[\frac{\pi_{\theta}(\mathbf{a}_{c,m} | \mathbf{q}_{c,m})}{\pi_{\theta_{\text{old}}}(\mathbf{a}_{c,m} | \mathbf{q}_{c,m})} A_{c,m}, \exp \left(\frac{\pi_{\theta}(\mathbf{a}_{c,m} | \mathbf{q}_{c,m})}{\pi_{\theta_{\text{old}}}(\mathbf{a}_{c,m} | \mathbf{q}_{c,m})} (1 - \epsilon, 1 + \epsilon) A_{c,m} \right) \right] \right\}, \\ \text{where } A_{c,m} = R(\mathbf{q}, \mathbf{a}_c) - \text{mean}(\{R(\mathbf{q}, \mathbf{a}_1), \dots, R(\mathbf{q}, \mathbf{a}_C)\}). \end{aligned}$$



Base model
already has 'aha'

Question(a): Find the least positive integer n such that $1/\sin 45^\circ \sin 46^\circ + 1/\sin 47^\circ \sin 48^\circ + \dots + 1/\sin 133^\circ \sin 134^\circ = 1/\sin n^\circ$.

Response
First, I notice that the left-hand side of the equation has the form $\sum_{k=1}^n \frac{1}{\sin a_k \sin a_{k+1}}$, where $a_1 = 45^\circ, a_2 = 47^\circ, \dots, a_{30} = 133^\circ$. I also notice that $\sin a_{k+1} = \sin(a_k + 2^\circ)$.
...
Simplifying, I get $\sin(3a) = 3 \sin a - 4 \sin^3 a$.
Aha! I can use this to get $\sin(a + 2b) = \sin((a + b) + b) = 3 \sin(a + b) \dots$
This looks messy, but I notice that it has the form $\sin a \cos b + \cos a \sin b$ again. This gives me an idea. Maybe I can use this identity repeatedly to simplify the left-hand side of the equation.
...

Figure 17: R1-Zero 的”aha moment”: 模型自发学会回溯¹⁷

1. CoT 长度随训练稳定增长——论文浪漫地解读为”模型学会了想得更深来解更难的题”
2. 涌现出”aha moment”与回溯——模型会写出”wait, I think I made a mistake...”这样的自我纠正

Dr.GRPO 的反驳

Dr.GRPO 作者对这两个现象提出了相当有力的反驳: (1) 长度增长可能只是有偏目标的副产物, 而非涌现能力; (2) 把 DeepSeek-V3 直接跑在数学题上, 它也会偶尔输出”aha, I can do this”——这未必是 RL 带来的新现象。近期证据让这些”涌现性”解释变得不再那么可信。

7.3 R1 的完整流水线

R1-Zero 是研究设置, 要部署一个真正能交付的强模型, 需要更完整的流水线。

¹⁷视频画面时间区间: 00:48:30 - 00:49:00。

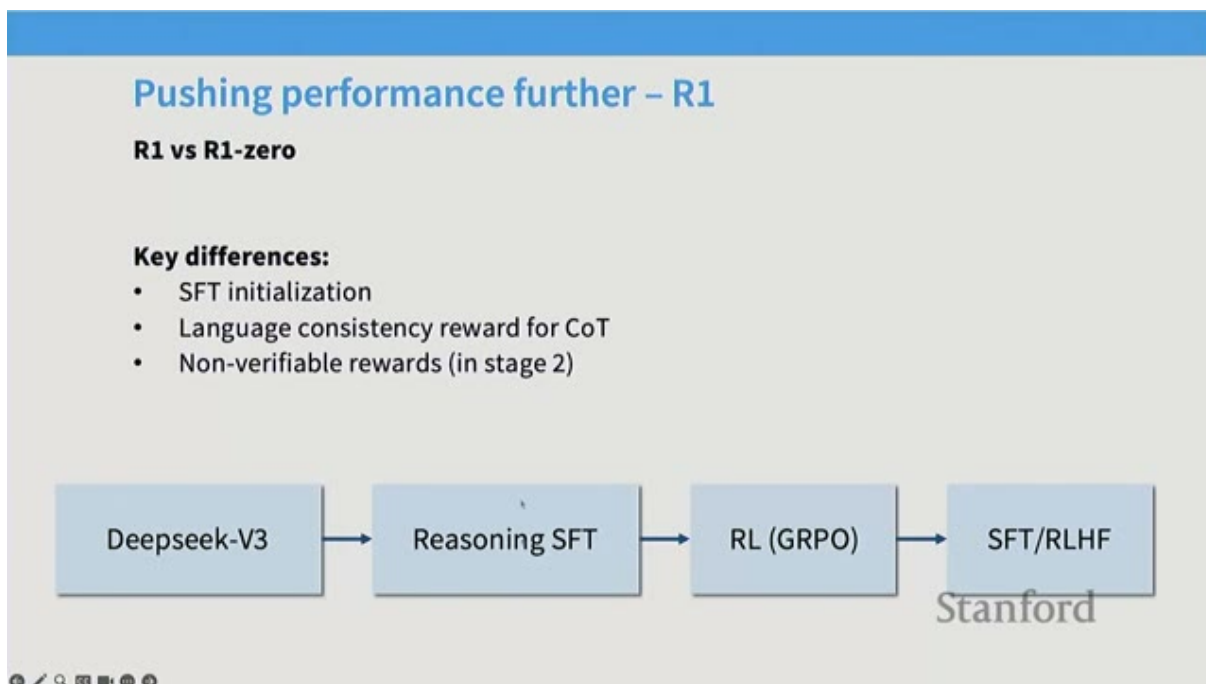


Figure 18: R1 完整流水线：长 CoT SFT $\rightarrow$ RL $\rightarrow$ 二次 SFT $\rightarrow$ RLHF¹⁸

1. 长 CoT SFT 初始化：用来源不明的长 CoT 数据做 SFT，让模型先“会”长思维链，再进 RL（论文对数据来源语焉不详）
2. 推理 RL：基本同 R1-Zero，额外加语言一致性奖励（language consistency reward），防止 CoT 中途切换语言
3. 二次 SFT：把推理数据 + 非推理数据（写作、证明等非可验证任务，用 self-judge）+ DeepSeek-V3 的 SFT 数据合并
4. RLHF：仍用 GRPO 做 RLHF，沿用 V3 的偏好训练流程

语言一致性奖励的必要性

Tatsu 强调一个有意思的现象：若不加这个奖励，模型在 RL 过程中 CoT 会语言混合——比如突然从英文切到中文（这正是互联网上用户抱怨“Grok 3 突然说中文”的根源）。这表明模型天然有语言混合倾向，需要额外奖励约束。代价是它略微牺牲了性能，但换取了 CoT 的可读性。

¹⁸视频画面时间区间：00:50:00 – 00:50:30。

7.4 R1 的性能

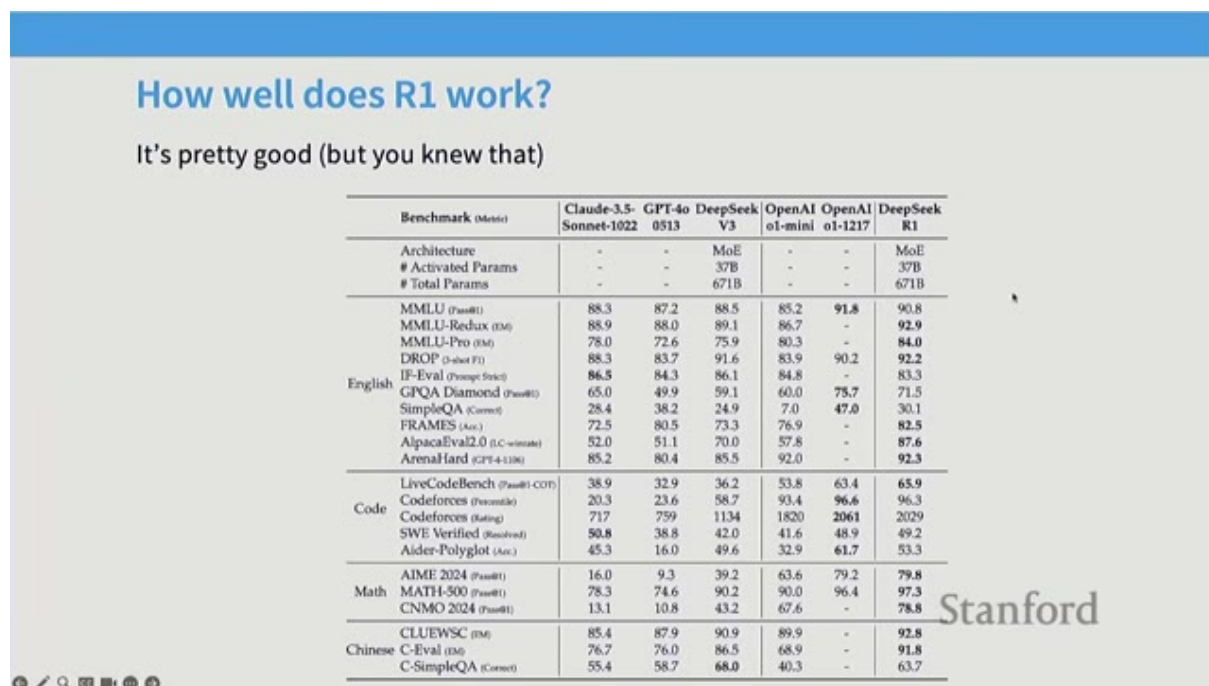


Figure 19: R1 在英文任务上几乎全面匹配 o1¹⁹

R1 的结果震撼了整个社区：英文任务全面匹配 o1，代码略差但非常接近。考虑到其配方的简洁性，这一结果几乎超出所有人的预期。

7.5 R1 的蒸馏

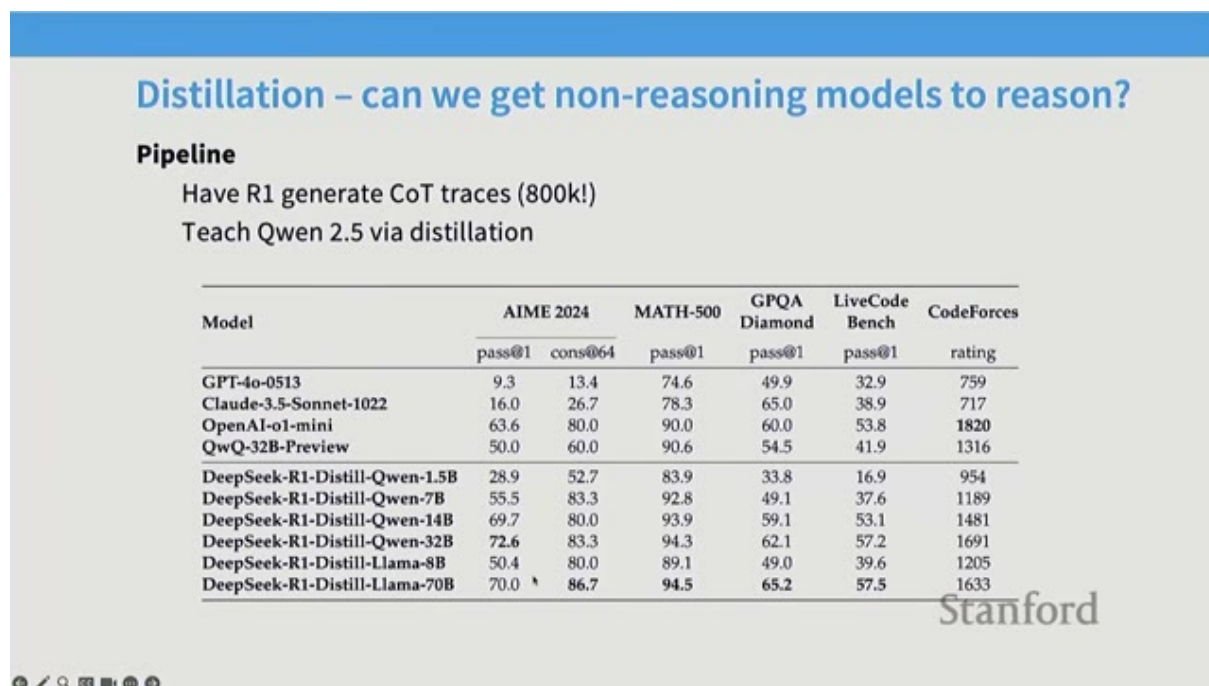


Figure 20: R1 蒸馏：百万级 CoT 灌给 Qwen，性能大幅提升²⁰

¹⁹ 视频画面时间区间：00:54:35 – 00:54:55。

R1 团队把 R1 生成的近百万条思维链作为 SFT 数据，fine-tune Qwen 等小模型。例如 32B 的 Qwen 基模 AIME 只有约 50%，蒸馏后能提升 25 个百分点以上。这再次印证：长 CoT 能力其实早已“潜伏”在基座里，只需少量数据就能“激活”。

Tatsu 自己组里也验证过：用 Gemini 2.0 Flash Thinking 的 1000 条长 CoT fine-tune Qwen 2.5，就能获得非常显著的数学 benchmark 提升。

7.6 R1 的三大科学贡献

1. 正面证据：纯结果奖励 + GRPO 就能训出强推理模型
2. 负面结果 1：PRM 不奏效——他们在 R1 上反复尝试 PRM 都没能复现 o1
3. 负面结果 2：基于搜索的方法（MCTS）也没奏效

两个“不奏效”的意义

DeepSeek 在 DeepSeek-Math 阶段对 PRM 信心十足（蓝线最优），但到 R1 却回归纯结果奖励；MCTS 等搜索方法也未能跑通。这两个负面结果让“结果奖励 + RL”成为推理模型的最强基线，至今未被撼动。

8 案例研究二：Kimi K1.5

Kimi K1.5 与 R1 几乎同期发布，达成了非常接近的结果，但用了不同的算法和工程细节。两者构成“趋同演化（convergent evolution）”——同一问题被两条路径独立攻下，对照阅读价值极高。

8.1 数据构造

Data curation + SFT

Data curation

- Standard curation across math-style settings, balancing topics

To achieve diverse coverage in the prompt set, we employ automatic filters to select questions that require rich reasoning and are straightforward to evaluate. Our dataset includes problems from various domains, such as STEM fields, competitions, and general reasoning tasks, incorporating both text-only and image-text question-answering data. Furthermore, we developed a tagging system to categorize prompts by domain and discipline, ensuring balanced representation across different subject areas (M. Li et al. 2023; W. Liu et al. 2023).

- Exclude multiple choice / true false (false positives)
- Select only examples that models fail on best-of-8

We adopt a model-based approach that leverages the model's own capacity to adaptively assess the difficulty of each prompt. Specifically, for every prompt, an SFT model generates answers ten times using a relatively high sampling temperature. The pass rate is then calculated and used as a proxy for the prompt's difficulty—the lower the pass rate, the higher the difficulty. This approach allows difficulty evaluation to be aligned with the model's intrinsic capabilities, making it highly effective for RL training. By leveraging this method, we can prefilter most trivial cases and easily explore different sampling strategies during RL training.

SFT – little description, just described as ‘prompt engineering’ (distillation?)

Stanford

Figure 21: Kimi K1.5 数据：best-of-8 难度过滤 + 域平衡²¹

²⁰ 视频画面时间区间：00:55:05 – 00:55:30。

²¹ 视频画面时间区间：01:01:55 – 01:02:25。

策略	做法
域平衡 排除 MCQ/判断题	用 LM 自动给数学题打学科标签，跨学科平衡保证多样性 太容易被 hack 或随机猜对，只保留可用 regex/LM 评判的短答案
难度过滤（关键）	用非推理 SFT 模型对每个题生成 10 个答案，计算 pass rate； 只保留 best-of-8 全错的题（只要 8 中有 1 对就视为太易而剔除）
SFT 数据	来源不详，只说做了 prompt engineering，显然蒸馏自他处

8.2 RL 算法：从 DPO 推导出的平方损失

Kimi 的 RL 算法从 DPO 的同一出发点开始，但走了一条不同的岔路。

Kimi RL

In kimi - reference based reward model, so the optimization problem is

$$\max_{\theta} \mathbb{E}_{(x, y^*) \sim \mathcal{D}} [\mathbb{E}_{(y, z) \sim \pi_{\theta}} [r(x, y, y^*)] - \tau \text{KL}(\pi_{\theta}(x) || \pi_{\theta_i}(x))],$$

RL algorithm inspired by DPO-type derivation.

$$r(x, y, y^*) - \tau \log Z = \tau \log \frac{\pi^*(y, z|x)}{\pi_{\theta_i}(y, z|x)}.$$

(Nonparametric assumption + solve for r)

$$L(\theta) = \mathbb{E}_{(x, y^*) \sim \mathcal{D}} \left[\mathbb{E}_{(y, z) \sim \pi_{\theta_i}} \left[\left(r(x, y, y^*) - \tau \log Z - \tau \log \frac{\pi_{\theta}(y, z|x)}{\pi_{\theta_i}(y, z|x)} \right)^2 \right] \right].$$

Use squared loss as a surrogate

Baselined policy gradient w/ regularization

$$\frac{1}{k} \sum_{j=1}^k \left(\nabla_{\theta} \log \pi_{\theta}(y_j, z_j|x) (r(x, y_j, y^*) - \bar{r}) - \frac{\tau}{2} \nabla_{\theta} \left(\log \frac{\pi_{\theta}(y_j, z_j|x)}{\pi_{\theta_i}(y_j, z_j|x)} \right)^2 \right).$$

Stanford

Figure 22: Kimi K1.5: 把最优策略等式做成平方损失²²

推导路径：从 KL 正则化的奖励最大化出发，做非参数假设得到 $r(x, y) = \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \log Z(x)$ （与 DPO 完全一致）；但 Kimi 不代入 Bradley-Terry，而是直接说“对最优策略这个等式成立”，于是把它做成平方损失驱动两边靠近：

$$\mathcal{L} = \left(r(x, y) - \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} - \log Z \right)^2$$

对它求梯度，形式上和 GRPO 非常像——策略梯度 + batch 内均值作为 baseline + 一个平方项作为正则化（替代 clip）。这说明：只要具备策略梯度、合理 baseline、某种正则化三件套，就能得到一个能跑的 RL 算法。

Kimi 起点的 KL 正则化目标可写为：

$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}} [r(x, y)] - \beta \mathbb{D}_{\text{KL}}(\pi_{\theta}(\cdot|x) || \pi_{\text{ref}}(\cdot|x))$$

- 第一项：在数据集 $\mathcal{D}$ 和策略 π_{θ} 下的期望奖励
- 第二项：KL 惩罚，防止策略漂离参考模型 π_{ref}
- β ：权衡探索与约束的系数——所有 RLHF/RLVR 算法的共同起点

²² 视频画面时间区间：01:03:30 - 01:04:00。

8.3 长度奖励：主动压缩 CoT

Kimi 团队比 R1 更早意识到推理成本的重要性——长 CoT 极其昂贵。所以他们主动设计长度奖励来压缩思维链。

Length control in Kimi

The kimi objective doesn't have the same GRPO length bias problem.
.. But they want to further compress the CoTs.

For each batch, they have a *length reward*

$$\text{len_reward}(i) = \begin{cases} \lambda & \text{If } r(x, y_i, y^*) = 1 \\ \min(0, \lambda) & \text{If } r(x, y_i, y^*) = 0 \end{cases}, \quad \text{where } \lambda = 0.5 - \frac{\text{len}(i) - \text{min_len}}{\text{max_len} - \text{min_len}}.$$

How should I interpret this?

- The lambda is in [0.5, -0.5], with longer sequences in a group being negative
- Correct answers are incentivized to be short
- Incorrect answers are incentivized to be shorter than the center of the range of rollouts
(They also only enable this later on in training, due to its effects on perf)

Stanford

Figure 23: Kimi K1.5 长度奖励：奖励短的、惩罚长的²³

$$\lambda = \frac{L_{\max} - |y|}{L_{\max} - L_{\min}} - 0.5$$

- $L_{\max}$ 、 $L_{\min}$ ：batch 内最长、最短回答
- $\lambda \in [-0.5, 0.5]$ ：越短越大
- 答对时：奖励 λ ，激励尽量短
- 答错时：也用 λ ，但压力较弱（鼓励不要太长但不强制最短）

长度奖励不能加得太早

Kimi 发现：如果训练早期就加长度奖励，RL 会停滞——模型什么都不会做，最优策略是把 CoT 拉到最短，然后陷入局部极小再也出不来。所以正确做法是先做一段无约束 RL，后期再加长度奖励。

8.4 课程与奖励工程的其他细节

- 难度标签：人工或 LM 给数据打难度标签，按“易 → 难”顺序训练
- 按成功率采样：成功率 p 越高的题采样权重越低； $p = 1$ 的题永不采样
- 代码奖励：用 ground-truth 解生成测试用例
- 数学奖励：用一个小奖励模型做答案等价性判断（而非简单的 regex/string match）——令人意外的是在“可验证”场景用模型判等，但因为本质是高级字符串匹配，准确率极高

²³ 视频画面时间区间：01:06:45 – 01:07:15。

多个奖励如何组合?

所有奖励 (accuracy、format、length、language consistency) 都是加权求和, 权重凭经验调以最大化下游性能。format 类奖励可看作 shaping——不是终点, 而是引导模型产出可用 CoT 的手段。

8.5 RL 基础设施

Kimi 论文是少有的详谈 RL 系统设计的推理模型论文。

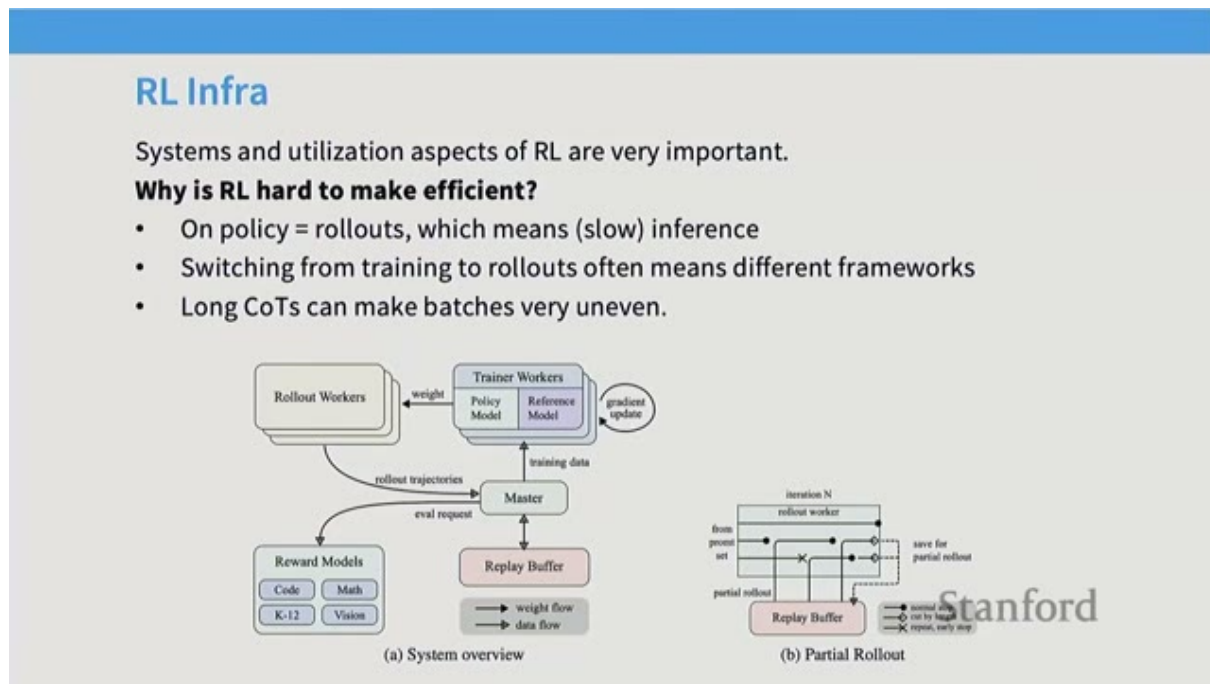


Figure 24: Kimi K1.5 RL 基础设施：训练 worker 与推理 worker 解耦²⁴

挑战	Kimi 的做法
rollout 昂贵	必须运行语言模型做推理，比预训练更难饱和 GPU
权重同步	训练 worker ↔ 推理 worker 之间要传模型权重
长 CoT	batch 长度极不均匀，需要巧妙调度
vLLM	推理用 vLLM；常“用 dummy 权重启动”，加载权重后再每轮拆毁以释放显存

vLLM 权重注入的工程痛点

目前把训练侧的 RL 权重“塞进”vLLM 还非常麻烦。虽然有实验性 NCCL collective API 可直接把权重灌进去，但参数文档不全、不够成熟。当前工业界的普遍做法是“dummy 权重启动 + hack 加载 + 每轮拆 vLLM 释放显存”。Tatsu 预期一年后这套基础设施会显著成熟，A5 也会用上简化版。

8.6 Kimi 的训练曲线

Kimi 展示了逐 RL 迭代的性能曲线：性能稳步上升，而回答长度平台化（不像 vanilla GRPO 那样无限增长）——这与 Dr.GRPO 的观察一致，因为 Kimi 的算法本身没有“鼓励变长”的偏差。

9 案例研究三：Qwen3

Qwen3 是三者中最新的，报告在课程结束前刚刚发布。它在 R1 基础上引入了若干新技巧，尤其在推理效率方向上有独到设计。

²⁴ 视频画面时间区间：01:09:30 – 01:10:15。

9.1 整体流水线

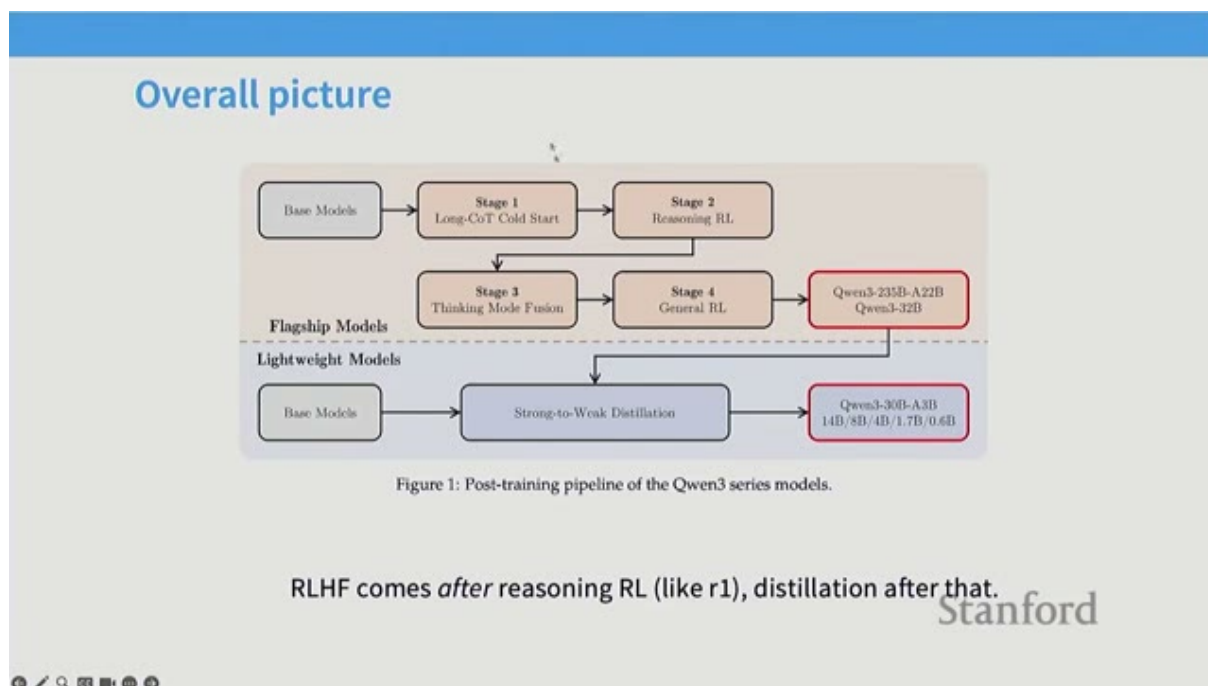


Figure 25: Qwen3 流水线：长 CoT SFT → 推理 RL → 思维模式融合 → 通用 RL²⁵

1. 长 CoT SFT：手工筛选（区分“猜对”与“真会”）
2. 推理 RL：在惊人的3,995 个样本上做 RL，仍获得可观增益
3. 思维模式融合（thinking mode fusion）：Qwen3 独创
4. 通用 RL（RLHF）：对齐一般指令

3995 个样本的样本效率

Qwen3 仅用约 4000 个 RL 样本就取得显著增益。这既说明 RLVR 的极高效率，也呼应了历史上“少量样本即可 instruction tune / 蒸馏长 CoT”的现象——但能少样本跑通并不等于不能继续 scale，背后机制目前仍不清楚。

9.2 思维模式融合：单模型双模式

这是 Qwen3 最有意思的创新。其目标是在同一个模型里同时拥有“思考模式”和“非思考模式”：训练后对模型做 SFT，让它学会根据 <think> / <no-think> 标签决定是否展开思维链。

²⁵ 视频画面时间区间：01:14:35 – 01:14:55。

Qwen 3 specific new stuff.

Thinking mode fusion – controlling the length of the CoT.

1. Mix non-thinking and thinking data with tags

Thinking Mode	Non-Thinking Mode
<pre>< im.start >user {query} /think< im.end > < im.start >assistant <think> {thinking.content} </think> {response}< im.end ></pre>	<pre>< im.start >user {query} /no.think< im.end > < im.start >assistant <think> </think> {response}< im.end ></pre>

2. Early stopping termination via a special string

implementing budget control over the model's thinking process. Specifically, when the length of the model's thinking reaches a user-defined threshold, we manually halt the thinking process and insert the stop-thinking instruction: "Considering the limited time by the user, I have to give the solution based on the thinking directly now.\n</think>.\n\n". After this instruction is inserted the model proceeds to generate a final response based on its accumulated reasoning up to that point. It is worth noting that this ability is not explicitly trained but emerges naturally as a result of applying Thinking Mode Fusion.

Figure 26: Qwen3 思维模式融合：think / no-think 标签 + 早停控制²⁶

- <think>: 正常长 CoT
- <no-think>: 直接给答案
- 附加能力：在思考过程中插入特殊字符串（如“考虑到用户的时间有限，我需要直接给出答案”）可以优雅地提前终止思维链，给出近乎正确的答案

test-time scaling 的新维度

思维模式融合让单一模型获得精细的“思考预算”控制：思考预算设到无穷就是原思考模型；设到某阈值就提前终止，性能从满血优雅退化。这是一种非常干净的test-time scaling 旋钮。

²⁶ 视频画面时间区间：01:17:15 – 01:17:45。

9.3 Qwen3 的消融：推理 RL vs 通用 RL 的权衡

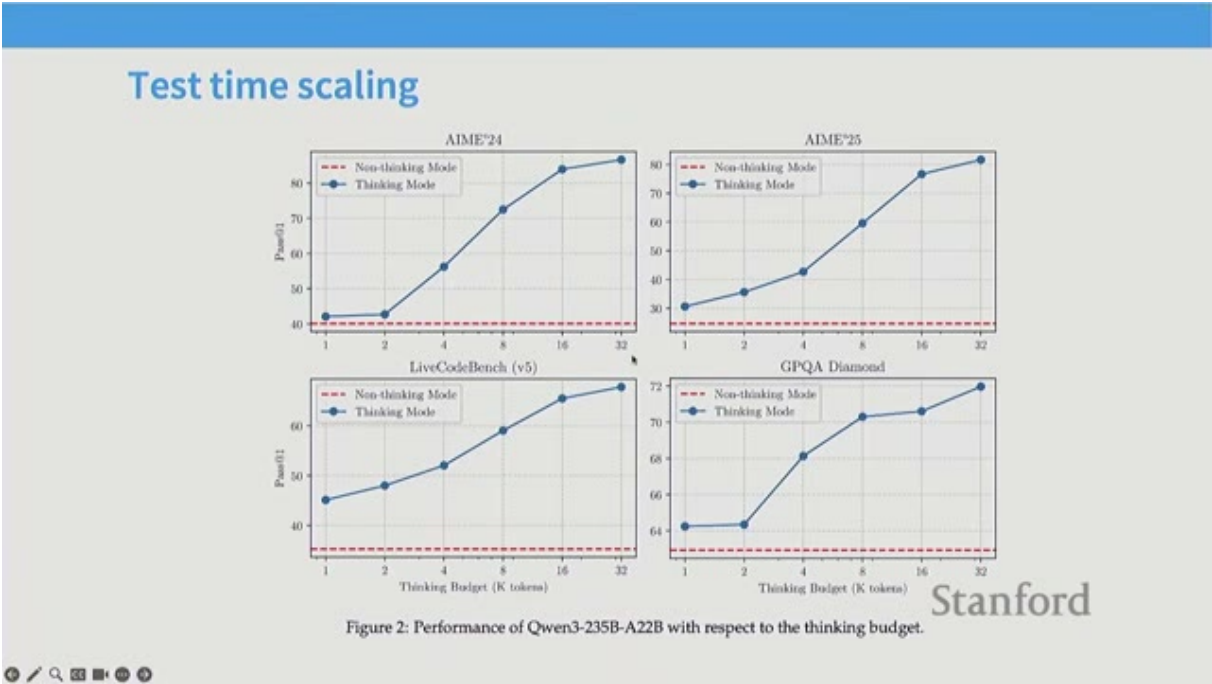


Figure 27: Qwen3 消融：推理 RL、思维融合、通用 RL 在不同任务上的影响²⁷

Qwen3 给出了推理 RL、思维融合、通用 RL 三阶段在 think / no-think 两种模式下的消融：

任务类型	推理 RL	通用 RL (RLHF)
通用 / 指令跟随	think 模式下都涨	继续涨
数学 / STEM (think)	通用 RL 伤害性能	—
数 学 / STEM (no-think)	通用 RL 帮助性能	—

推理与通用能力的张力

Qwen3 的消融揭示了一个真实存在的权衡：优化”通用指令跟随”和优化”数学/代码推理”在 think 模式下是冲突的——通用 RL 会拉低推理性能。如何在同一模型里同时做好两类任务，是后续模型要攻克的开放问题。

²⁷ 视频画面时间区间：01:18:30 – 01:19:00。

10 总结与展望

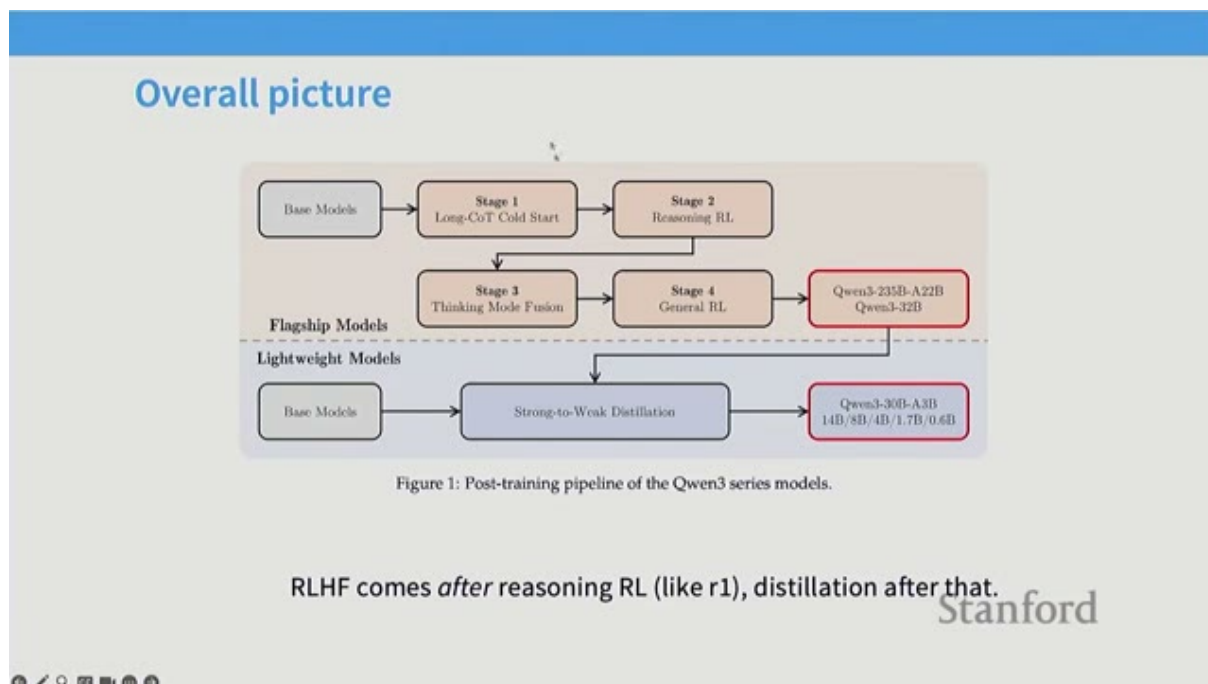


Figure 28: 三个案例的共同骨架：SFT → 推理 RL → （融合）→ 通用 RL²⁸

10.1 核心结论

本讲的核心信息

1. RL 很强大，但靠“对噪声成对偏好不断 hill-climb”的 RLHF 走不远
2. 解法：选择无法 reward hacking 的领域（数学、代码），把 RL 武器全搬过来——这是 RLVR 的根本动机
3. 算法：GRPO 极其简单——策略梯度 + 组内 baseline 即可。Dr.GRPO 进一步揭示了两个被忽视的偏差
4. 工程：三篇论文展示了 SFT → RL → RLHF 的成功配方，以及各自的关键实现技巧

10.2 三个案例的对比

维度	DeepSeek-R1	Kimi K1.5	Qwen3
RL 算法	GRPO (clip)	平方损失正则 (类 DPO)	GRPO 变体
价值模型	无	无	无
奖励	accuracy + format + 语言一致	accuracy + length + 课程	accuracy + 思维融合
长度控制	无 (被动增长)	有 (length reward)	有 (thinking budget)
PRM	否	否	否
搜索	否 (MCTS 失败)	否	否
数据规模	未公开	未公开	3,995 样本

²⁸ 视频画面时间区间：01:14:35 – 01:14:55。

三者的“趋同演化”

R1 与 Kimi K1.5 几乎同期独立完成，Qwen3 在其后吸收两者经验。三者都放弃了 PRM 与搜索、都用纯结果奖励 + 无价值模型的 RL、都遵循“SFT → 推理 RL → 通用 RL”骨架。这种趋同强烈暗示：当前推理模型的最优配方已基本收敛，差异主要在数据、长度控制与基础设施等工程层面。

10.3 贯穿全讲的几条主线

1. 结果奖励胜过过程奖励与搜索：DeepSeek 在 Math 阶段信 PRM，到 R1 放弃；MCTS 也未跑通。至今纯结果奖励 + RL 仍是最强基线。
2. 基线（baseline）是 RL 的灵魂：无论是 GRPO 的组内均值、Kimi 的 batch 均值，还是 PPO 的价值函数，本质上都是降低梯度的方差。
3. 长 CoT 不是“想得更深”，可能是“目标有偏”：Dr.GRPO 和 Kimi 的曲线都暗示，长度增长部分来自算法偏差而非内在需求。
4. 推理与通用能力的张力真实存在：Qwen3 消融明确显示 think 模式下通用 RL 会伤害推理。这是未来模型的关键开放问题。
5. RL 基础设施远未成熟：vLLM 权重注入、rollout 调度、worker 解耦都还有大量工程空间。

作业落地

A5 将让你亲手实现 GRPO。你会遇到本讲提到的几乎所有细节：组内 z-score、除以标准差加 epsilon、KL 的 per-token vs per-sequence、vLLM 权重同步、外层训练循环与内层 rollout 的解耦。把这些细节跑通，你就真正理解了推理模型是如何被训练出来的。